

0.51. Jusqu'au plus grand : une boucle

Une boucle `for` fait grandir la taille de 1 à 8 : huit carrés emboîtés, jusqu'au plus grand que l'écran tienne.

1. Ce qu'il faut comprendre

Le 0.50 répétait la même ligne, avec une taille qui augmente de 1 à chaque fois. Quand seul un nombre change, et toujours de la même façon, on écrit la ligne **une seule fois**, dans une **boucle**.

Ce qui est nouveau ici : la taille est une **variable**, `taille`, que la boucle `for` fait passer par 1, 2, 3... jusqu'à 8. À chaque tour de boucle, `carre` reçoit la taille du moment.

`taille <= 8` veut dire « tant que la taille est plus petite **ou égale** à 8 » : la boucle fait donc aussi le tour de taille 8, puis s'arrête.

Pourquoi 8 : c'est le plus grand carré que l'écran tienne. Autour de (10, 8), il va de la colonne 2 à 18 et de la ligne 0 à 16. Le centre a été choisi pour ça (voir le 0.49).

La vitesse, 50 ms par pas, est plus rapide : huit carrés à 250 ms dureraient plus d'une minute. Elle s'écrit toujours en clair ; la taille, elle, peut être une variable.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <carre>` → [0.33.3. La fonction `carre\(\)` — tourner en carré](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1368 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage — la mise en route de la console
carre()         440 o ← main() — une fonction fournie par le langage
main()          57 o ← démarrage — ta fonction
VBlank          8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage   28 o ← image() — attend l'image suivante
AttendreVBlank  16 o ← démarrage — attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases — attend de pouvoir écrire une case
AdresseCase     10 o ← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() — efface des cases
CopierTuiles    18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage — vide les deux cartes de l'écran
Tuiles          704 o ← CopierTuiles — la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 4      52 o sur place, dans carre() + AdresseCase (10 o)
```

```
effacer() × 3    45 o sur place, dans carre() + AdresseCase (10 o), EffacerCases (12 o)
image()  × 4    12 o sur place, dans carre(), main() + AttendreImage (28 o)
Tes fonctions appelées :
carre()  × 1    31 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <carre>      // une tuile qui tourne en carré
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // La boucle, morceau par morceau :
    //
    // for (uint8_t taille = 1; taille <= 8; taille++)
    //     |           |           |
    //     |           |           +-- après chaque tour : taille + 1
    //     |           +----- tant que taille ≤ 8 (8 compris)
    //     +----- on commence à la taille 1
    //
    // tour 1 : carre(10, 8, ALPHABET[0], 1, 1, 50, 1);    3 × 3
    // tour 2 : carre(10, 8, ALPHABET[0], 2, 1, 50, 1);    5 × 5
    // ...
    // tour 8 : carre(10, 8, ALPHABET[0], 8, 1, 50, 1);    17 × 17, le plus grand
    //
    // Le plus grand, autour du centre (10, 8) :
    // colonnes 2 à 18, lignes 0 à 16 : il tient entier dans l'écran
    // (20 colonnes de 0 à 19, 18 lignes de 0 à 17).
    for (uint8_t taille = 1; taille <= 8; taille++) {
        carre(10, 8, ALPHABET[0], taille, 1, 50, 1); // 50 ms par pas : plus vite
    }

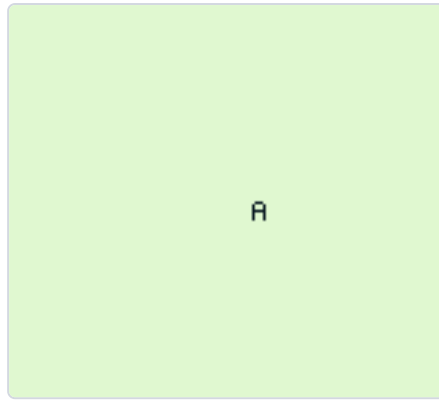
    while (true) {
        image(); // les huit carrés sont faits : le A est au centre
    }
}
```

3. Ce qui se passe, étape par étape

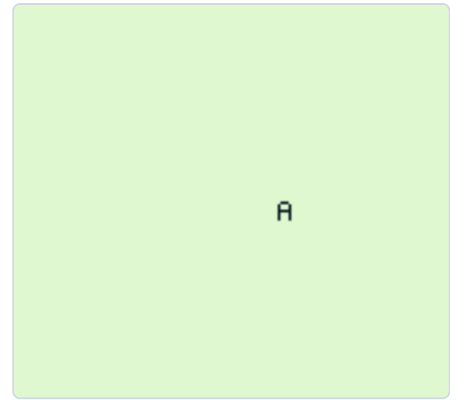
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



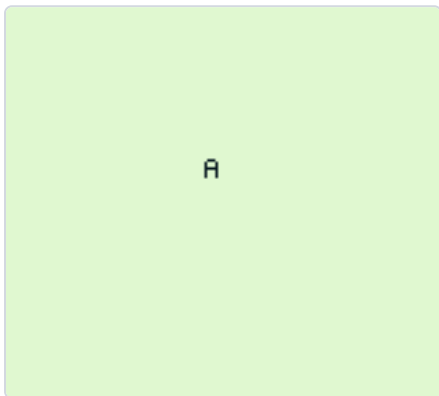
la toute première image — la console vient de démarrer — image 1



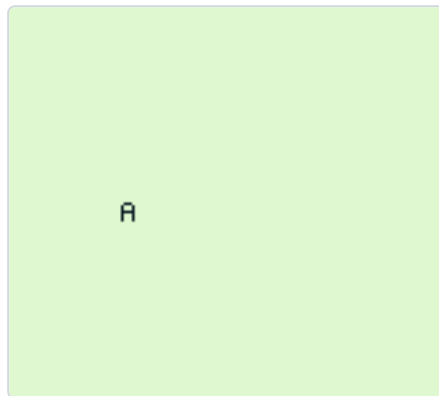
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Huit tours autour du milieu de l'écran, de plus en plus grands, jusqu'à un carré qui touche presque les bords.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les coins grandissent : (12, 10), (14, 12), (18, 16)
- ✓ le plus grand va jusqu'en haut à gauche (2, 0)
- ✓ jamais hors de l'écran : ni colonne 19, ni ligne 17
- ✓ le A revient au centre (10, 8) (10,8)

Ensuite : **0.51.1. Jusqu'au plus grand : une boucle — de base, ailleurs** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++